Data Analysis and Visualization: course project, Phase 3

Wikipedia Knowledge Integrity Lakehouse for Pakistan's Languages

Phase 3: Final Architecture, Gold Model and FinOps Report

Team	Zuraiz Anjum (23L-2547) Zoraiz Rizwan (23L-6008)
Course	Data Analysis and Visualization
Instructor	Amir Iqbal
Repository	github.com/zuraiz-anjum/pk-wiki-integrity-lakehouse
Platform	Databricks (Spark, Delta Lake, Unity Catalog); costs priced for Azure Databricks
Dashboard	Power BI Desktop, Knowledge Integrity Monitor
Data	Wikimedia MediaWiki history dumps (full load) and the MediaWiki recent changes API (incremental)
Date	26 October 2026

This report describes the finished lakehouse that tracks edits, page events, reverts and suspected vandalism on the Urdu, Punjabi (Shahmukhi), Sindhi and Pashto Wikipedias. It covers the end-to-end architecture, the Gold star schema that feeds Power BI, the design decisions we made and why, how Spark executes the main Gold aggregation, and what the pipeline costs to run today and at enterprise scale.

Contents

1 Introduction . . . 3

2 End-to-end architecture . . . 3

2.1 Overview . . . 3

2.2 Layers and tables . . . 5

2.3 Orchestration . . . 5

3 Gold dimensional model . . . 5

3.1 Star schema . . . 5

3.2 Data dictionary summary . . . 7

4 Decision log . . . 8

4.1 Partition columns . . . 8

4.2 Schema drift handling . . . 9

4.3 PII management . . . 10

4.4 Idempotent MERGE, SCD Type 2 and incremental watermarking . . . 10

4.5 Gold MERGE strategy and table optimisation . . . 11

4.6 A surrogate key bug, found and fixed . . . 12

5 Spark UI and DAG analysis . . . 13

5.1 Query Profile of a Gold MERGE . . . 13

5.2 Physical plan of the agg_wiki_daily build . . . 15

5.3 How the profile was captured . . . 16

6 FinOps . . . 17

6.1 Pricing basis . . . 17

6.2 Typical daily incremental run . . . 17

6.3 Enterprise extrapolation: hourly runs for a year . . . 18

6.4 Optimisations . . . 19

7 Conclusion and future work . . . 20

8 References . . . 20

1 Introduction

Wikipedia has active editions in four of Pakistan's languages: Urdu (`urwiki`), Western Punjabi in Shahmukhi script (`pnbwiki`), Sindhi (`sdwiki`) and Pashto (`pswiki`). Small volunteer communities look after them, and nobody has a clear, regularly updated picture of how healthy they are. Our project builds that picture. A Spark pipeline on Databricks follows every edit, new page, deletion and revert, and a Power BI dashboard turns the result into answers for the communities and for groups that work on regional languages.

Phase 1 was the proposal. In Phase 2 we built the Bronze and Silver layers, the audit tables and the daily job, and ran them on Databricks. Phase 3 adds the Gold dimensional model and the dashboard. This report is the final technical record: what the system looks like end to end, why we built it this way, how Spark runs the heaviest Gold query, and what it would cost to run for real.

Table 1. The five business questions and the Gold tables that answer them.

#	Business question (from the Phase 1 proposal)	Main Gold source
Q1	Are the four wikis growing, both in new articles and in active editors?	agg_wiki_daily, agg_editor_monthly
Q2	When someone vandalises a page, how long does it take to undo it? Does it differ for registered, temporary and IP editors?	agg_revert_latency, fact_revision
Q3	Which pages are the most fought over, and what are they about?	agg_edit_wars, dim_page
Q4	How much of each wiki is written by bots rather than people?	agg_wiki_daily, agg_editor_monthly
Q5	Did anonymous editing or vandalism change after IP editing was replaced by temporary accounts?	fact_revision, agg_revert_latency, dim_wiki

A note on the platform. All of our runs so far used Databricks Free Edition, which is serverless only and uses Unity Catalog. That keeps the project free, so there is no real bill. Databricks still records how many DBUs every run used, and Section 6 prices those measured DBUs at Microsoft's published Azure Databricks rates, so the numbers mean something for a paying deployment.

2 End-to-end architecture

2.1 Overview

Figure 1 shows the whole system. Two Wikimedia sources feed it. The monthly MediaWiki history dump [9] is the full load: one bzip2 TSV per wiki with 78 columns and no header. The MediaWiki recent changes API [10] is the incremental load: our extractor (`ingestion/ingest_wikimedia.py`) asks each wiki for everything since its saved watermark and writes JSON lines with a small envelope (`_wiki`, `_source`, `_ingested_at`, `_window_start`). Both land in a Unity Catalog volume in Hive-style folders, so a folder name already says which load, which snapshot or day, and which wiki a file belongs to.

Bronze keeps the data as it arrived and only adds lineage (`record_hash`, `source_file`, `batch_id`, `load_timestamp`) and drift markers. Silver casts every field, unifies the dump and the API into one model, masks personal data, sends rows that fail a check to quarantine, and keeps SCD Type 2 history for pages and editors. Gold reshapes Silver into a star schema with two event facts, four aggregate facts and five dimensions. Power BI connects through a Databricks SQL warehouse and reads the Gold tables plus the three ops tables for its pipeline health page. Every unit of work in every layer writes one row to `pkwiki_ops.pipeline_execution_logs`.

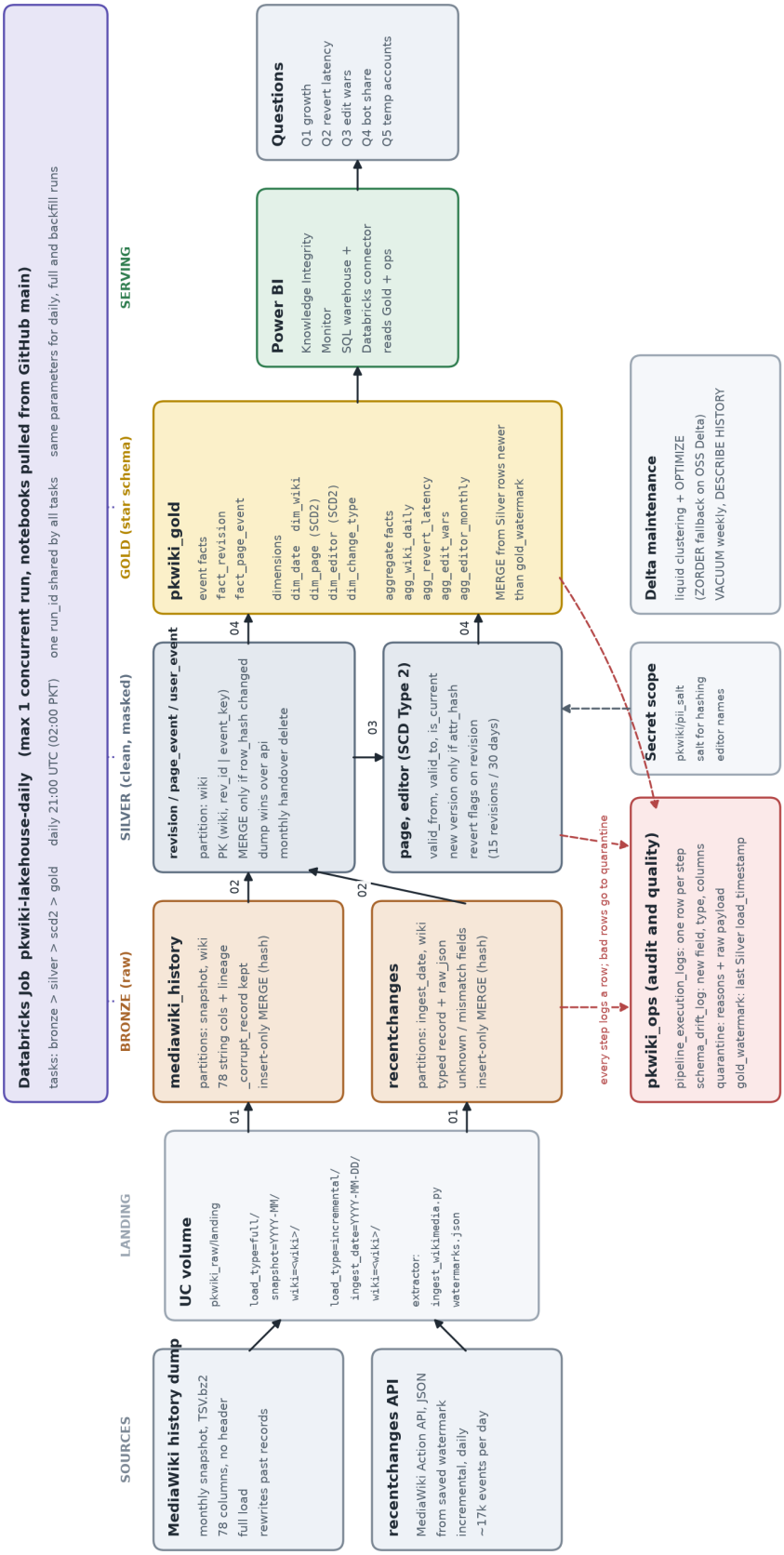


Figure 1. End-to-end architecture (page turned sideways). Solid arrows are data flow, numbered by the notebook that does the step (01 to 04). Dashed red arrows are audit and quarantine writes. The Databricks job runs the four notebooks in order.

2.2 Layers and tables

Table 2. Every table in the lakehouse with its grain, layout and write pattern.

Layer	Table	Grain	Physical layout	Write pattern
Bronze	mediawiki_history	one dump line	partitioned by snapshot, wiki	insert-only MERGE on (snapshot, wiki, record_hash)
Bronze	recentchanges	one API line	partitioned by ingest_date, wiki	insert-only MERGE on (ingest_date, wiki, record_hash)
Silver	revision	one edit	partitioned by wiki	MERGE on (wiki, rev_id); update only when row_hash changes; dump wins over API
Silver	page_event, user_event	one log event	partitioned by wiki	MERGE on (wiki, event_key), same rules
Silver	page, editor	one version (SCD2)	partitioned by wiki	staged SCD2 MERGE; new version only when attr_hash changes
Gold	dim_date, dim_wiki, dim_change_type	day / wiki / change kind	small, unpartitioned	seeded with insert-if-missing MERGE
Gold	dim_page, dim_editor	one version (SCD2)	liquid clustering	MERGE from Silver rows newer than the Gold watermark
Gold	fact_revision, fact_page_event	one edit / one page event	liquid clustering	MERGE from Silver rows newer than the Gold watermark
Gold	agg_* (4 tables)	wiki x day, wiki x month, page x day	small, unpartitioned	recompute touched days or months, MERGE, delete groups that vanished
Ops	pipeline_execution_logs	one step of one run	unpartitioned	append only, on purpose
Ops	schema_drift_log	one field per batch	unpartitioned	MERGE on its key
Ops	quarantine	one rejected row	partitioned by source_table	insert-only MERGE
Ops	gold_watermark	one Gold table	unpartitioned	MERGE after each Gold write

2.3 Orchestration

One Databricks job, `pkwiki-lakehouse-daily` (`jobs/pkwiki_pipeline.json`), runs the notebooks as dependent tasks: `bronze`, `silver`, `scd2` and `gold`. The definition has no cluster block, so the tasks run on serverless compute. Its `git_source` points at the `main` branch, which means the code that runs is exactly the code that was reviewed and merged. The schedule is 21:00 UTC (02:00 in Pakistan) and the job is created paused, so it only runs daily once we unpause it. We set `max_concurrent_runs` to 1. Two runs MERGEing into the same tables at the same time would not corrupt Delta, but one of them would fail with a concurrent-write conflict, and there is no reason to allow it.

All tasks receive the same job parameters (`load_type`, `snapshot`, `start_date`, `end_date`, `wikis`, `bronze_mode`, `run_id`). With no dates the run is the standard daily one. With a date range it is a backfill, and with `load_type=full` it loads a monthly snapshot. `run_id` defaults to `{{job.run_id}}`, so all steps of one run share an id in the audit log. If one wiki fails in Bronze, the other wikis still load, the failure is logged, and the notebook ends with an error so the job turns red and the later tasks do not start.

3 Gold dimensional model

3.1 Star schema

Gold is a star schema built for Power BI. There are two event facts at the finest grain, four aggregate facts for the heavier calculations, and five shared dimensions. `fact_revision` has one row per edit and carries the measures most questions need: byte changes, revert flags, `seconds_to_revert` and a vandalism-suspect flag (the edit was reverted within 24 hours by a different editor). `fact_page_event` has one row per page log event such as a creation, move, deletion or protection. The aggregate facts are pre-computed for the dashboard. `agg_edit_wars` keeps every page and day with at least two reverts, so the report can rank

contested pages, and its `is_edit_war` flag marks the stricter Phase 1 definition of an edit war: three or more reverts by two or more different editors within the day. On Databricks, 6 of the 18 contested page-days meet that stricter rule.

Some conventions hold across the model. Surrogate keys of the SCD2 dimensions and the event facts are `bigint` values from `xxhash64` of the natural key with the sign bit cleared, so a real key is never negative and can never be mistaken for an unknown member (Section 4.6). The same row always gets the same key, and a rerun never renumbers anything. Date keys are `yyyymmdd` integers and month keys are the date key of the first day of the month, so both join to `dim_date`. `dim_date` plays two roles: `date_key` is the UTC day and `pkt_date_key` is the Pakistan day, kept for the hour-of-week heat map that Phase 1 listed as a stretch goal. A fact joins to the version of the page or editor that was valid at the time of the edit, not the current one, so a page renamed last week still shows its old title on old edits. When no version matches, the fact points at the unknown member of its wiki (keys `-1` to `-4`), which exists in both SCD2 dimensions, so no fact row is ever dropped by an inner join. The next run retries those rows and repoints them once the dimension has caught up. On Databricks, 23.09 percent of `fact_revision` rows point at an unknown page and none at an unknown editor. All of those rows are dump revisions from 2002 to 2006 of pages that were later deleted: the dump keeps the edit but not the page, so the unknown member is the honest answer. Every revision that has a page id finds its page.

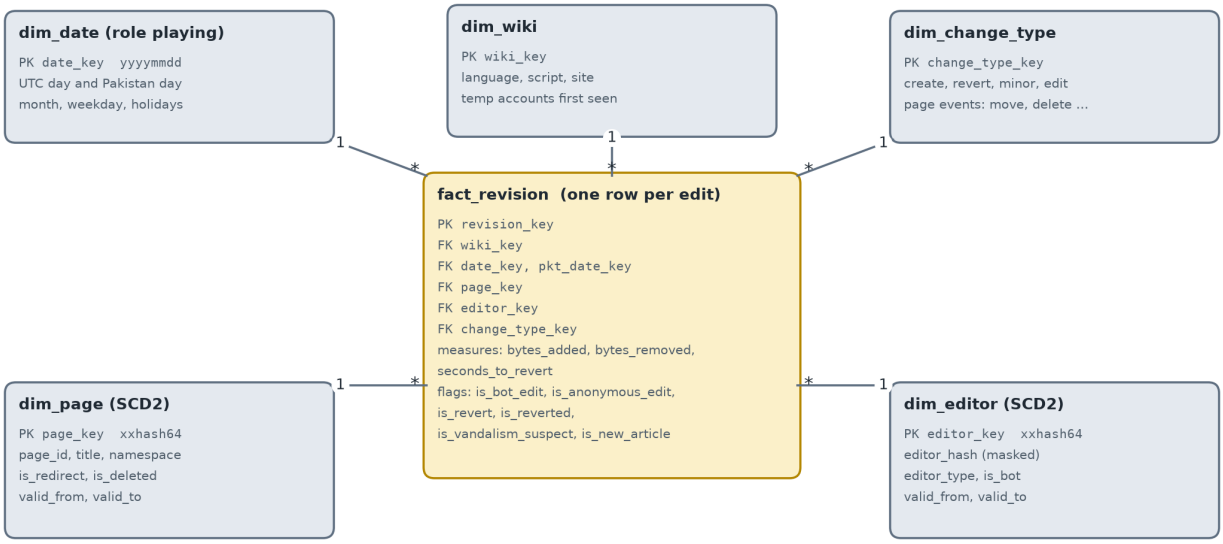


Figure 2. The star around fact_revision. Each fact row points at exactly one row in each dimension; dim_date is used twice (UTC day and Pakistan day).

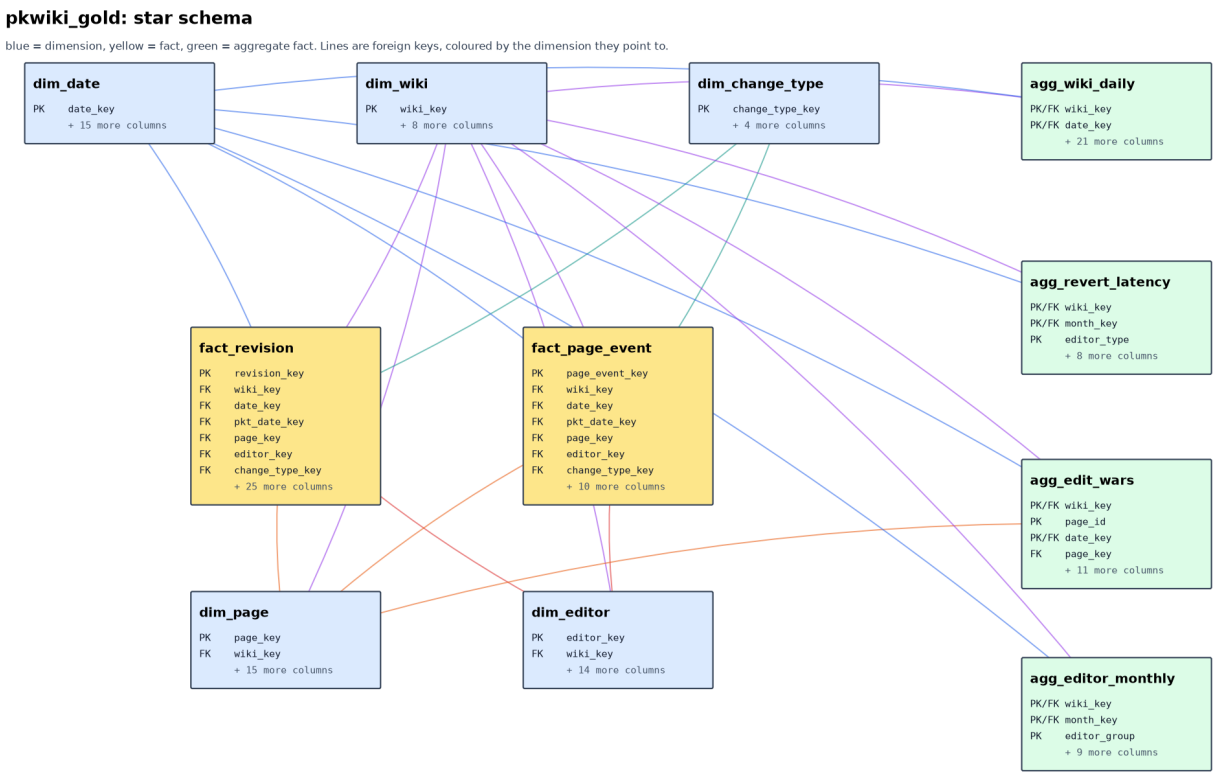


Figure 3. ER diagram of the whole Gold schema: five dimensions (blue), two event facts (yellow) and four aggregate facts (green). Lines are foreign keys, coloured by the dimension they point to.

3.2 Data dictionary summary

Table 3 summarises every Gold table. The full column list with types and descriptions is in `docs/gold_data_dictionary.md`, which is generated from the same schema definitions that create the tables, so the two cannot drift apart.

Table 3. Gold data dictionary, summary level.

Table	Type	Grain	Primary key	Main columns
dim_date	dimension	calendar day, 2001 to 2030	date_key	date, year, quarter, month, year_month, day_of_week, is_weekend, is_pk_public_holiday
dim_wiki	dimension	wiki (4 rows)	wiki_key	wiki, language_name, language_native, script, site_url, temp_accounts_first_seen_ts
dim_page	SCD2 dimension	page version	page_key	page_id, page_title, namespace_name, is_content_page, is_redirect, is_deleted, valid_from, valid_to, is_current
dim_editor	SCD2 dimension	editor version	editor_key	editor_hash (masked), editor_type, is_bot, is_anonymous, registration_ts, valid_from, valid_to, is_current
dim_change_type	dimension	kind of change (12)	change_type_key	entity (revision or page_event), change_type, description
fact_revision	event fact	one edit	revision_key	FKs to all five dims; bytes_added, bytes_removed, seconds_to_revert; is_bot_edit, is_anonymous_edit, is_new_article, is_revert, is_reverted, is_vandalism_suspect
fact_page_event	event fact	one page log event	page_event_key	FKs to all five dims; event_key, log_id, event_ts, event_ts_pkt
agg_wiki_daily	aggregate	wiki x day	wiki_key, date_key	edits, bot_edit_share, anonymous_edits, new_articles, pages_deleted, active_editors, reverts, vandalism_suspects, median_seconds_to_revert

Table	Type	Grain	Primary key	Main columns
agg_revert_latency	aggregate	wiki x month x editor type	wiki_key, month_key, editor_type	reverted_edits, median, p90 and average seconds to revert
agg_edit_wars	aggregate	contested page x day (2+ reverts)	wiki_key, page_id, date_key	revert_count, distinct_reverters, reverted_edits, distinct_editors, span_minutes, is_edit_war (3+ reverts by 2+ editors, the Phase 1 rule; true for 6 of 18 rows)
agg_editor_monthly	aggregate	wiki x month x human/bot	wiki_key, month_key, editor_group	editors, active (5+ edits), very active (100+), new_editors, returning_next_month

No raw usernames or IP addresses reach Gold. Editors are only known by the salted hash from Silver, bot accounts keep their public names, and edit summaries are not copied to Gold at all because the dashboard never needs them.

4 Decision log

This section records the main design choices and the reasons behind them. Table 4 is the short version; the subsections give the detail and, where it matters, the alternative we rejected.

Table 4. Decision summary.

Area	What we chose	Main reason
Partitioning	Bronze by (snapshot or ingest_date, wiki); Silver by wiki; Gold not partitioned but liquid clustered	Partitions match the unit of loading and of deletion; Gold is too small for partitions to help
Schema drift	Explicit schemas, all-string parse plus try_cast, drift log and quarantine; schema evolution only through a code change, in Bronze and in Gold	Never lose data and never fail four wikis for one bad file
PII	Salted SHA-256 for editor names, IPs and temporary accounts; regex masking of summaries; salt in a secret scope	Distinct-editor counts still work but nobody can be identified
Idempotency	Every write is a MERGE with a change condition, except the append-only audit log	A failed or repeated run can simply be run again
History	SCD Type 2 for page and editor, staged MERGE	Old titles and old editor types stay correct on old edits
Incremental	Extractor watermark per wiki, two newest ingest days in Silver, load_timestamp watermark for Gold	Late files and dump restatements are picked up without full rebuilds
Gold refresh	MERGE for facts and dimensions; recompute only touched days for aggregates	Cost scales with the day's changes, not with history
Layout	Liquid clustering plus OPTIMIZE on Databricks, ZORDER as the fallback on open source Delta	Data skipping on the dashboard filters without small-file problems
Surrogate keys	xxhash64 of the natural key with the sign bit cleared; negative keys only for unknown members	Stable keys without a lookup table; fixes a bug that sent half the facts to unknown members

4.1 Partition columns

Bronze. `bronze.mediawiki_history` is partitioned by `snapshot` and `wiki`, and `bronze.recentchanges` by `ingest_date` and `wiki`. These are exactly the folder levels in the landing volume, and each (day or snapshot, wiki) pair is one unit of work in the code. Three things follow from that. The insert-only MERGE includes the partition in its condition (`ON t.record_hash = s.record_hash AND t.ingest_date = '2026-09-27' AND t.wiki = 'urwiki'`), so Delta only reads the files of that one partition to find duplicates. A rebuild after a schema change is a cheap `DELETE` of whole partitions followed by a normal load. And a broken file for one wiki cannot touch the data of another.

There is a cost we accept. Pashto and Punjabi produce only a handful of API events per day (4 and 6 in our sample day), so their daily partitions hold tiny files. At our volume this is harmless, and a weekly `OPTIMIZE` compacts them. At a much larger scale we would move Bronze to liquid clustering on the same columns and

keep the partition filter as a clustering filter.

Silver. The Silver tables are partitioned by `wiki` only. Every Silver MERGE key starts with `wiki`, the monthly handover deletes by wiki, and almost every dashboard query filters by wiki. We considered partitioning by `rev_date` and rejected it. Revisions go back to 2002, so date partitions would mean thousands of small partitions, and the MERGE keys `(wiki, rev_id)` do not contain a date, so the MERGE could not prune by date anyway.

Gold. Gold has no Hive-style partitions. Even with the full dumps loaded, the largest Gold table is a few million rows, and Databricks recommends against partitioning tables much below a terabyte because it mostly produces small files [5]. Instead the two facts and the two SCD2 dimensions use liquid clustering on the columns that dashboard filters and joins use (date_key, wiki_key and page_key on the facts, wiki_key and page_id on dim_page, wiki_key and editor_hash on dim_editor). Clustering keys can be changed later without rewriting the table, which partitions cannot. `ops.quarantine` is partitioned by `source_table` because a replay always works on one source at a time.

4.2 Schema drift handling

We do not control the Wikimedia formats, so the rule is: keep the data, log the change, quarantine what does not fit, and never fail the whole run because of one bad file. Nothing uses `inferSchema`. Detection happens in Bronze:

- The dump is read with an explicit 78-column string schema in `PERMISSIVE` mode. A line that does not split into 78 fields goes to `_corrupt_record` and the drift log gets a `column_count` entry such as "79 fields instead of 78".
- Each API line is parsed first with every field as a string, then each field is `try_cast` to its declared type. A field that had a value before the cast and is null after it had the wrong type, so we know exactly which field broke (`type_mismatch`).
- Keys in the JSON that are not in our schema are found with `json_object_keys` and stored in `unknown_fields` (`new_field` in the drift log). The full original line stays in `raw_json`, so nothing is lost.
- Lines that are not valid JSON at all are kept with a null record and logged as `malformed_json`.

Silver then quarantines any row that is malformed, has a type mismatch, has a wiki that does not match its folder, or lacks a field we depend on (timestamp, revision id, page id, byte count). `ops.quarantine` keeps the list of reasons and the raw line, so a row can be replayed once the schema is fixed. We proved this with a small set of deliberately broken Pashto files in `sample_data/drift_demo`, run into a throwaway schema prefix so the real tables stay clean:

Table 5. Drift demo results, identical on the local runner and on Databricks.

Broken input	What the pipeline did
New field rc_quality_score in an API record	Row loaded normally; field kept in raw_json; new_field row in the drift log
revid: "abc123" and oldlen: "big"	type_mismatch entries for revid and oldlen; both rows quarantined
JSON line cut off halfway	malformed_json in the drift log; row quarantined
Two dump rows with a 79th column, one with only 40	column_count entries; 3 rows quarantined
revision_id = not_a_number in the dump	Quarantined in Silver as "bad revision_id"
Total	**6 drift log rows, 7 quarantined rows; the good rows from the same files (1 dump revision, 3 API revisions, 2 page events) reached Silver; every step SUCCESS; a second run added nothing**

Handling a new field properly is a deliberate step. We add it to `pipeline/schemas.py`. On the next run `evolve_schema()` compares the incoming DataFrame with the table and runs `ALTER TABLE ... ADD COLUMNS` for anything missing, nested fields included. That is the same effect as `mergeSchema`, done by hand

because we write with MERGE. To fill the new column for past days we rerun those days with `bronze_mode=rebuild`, which re-reads the raw files. We chose not to switch on automatic schema merging. With it, a single malformed record carrying a misspelt key would silently add a permanent column to a table. We would rather see the field in the drift log and decide.

Gold uses the same mechanism for its own changes. `merge_gold()` calls `evolve_schema()` before every MERGE, so a column we add to a Gold table definition appears on the next run without dropping or recreating the table. We used this ourselves late in Phase 3: `agg_edit_wars` gained `reverted_edits` and `is_edit_war` when we widened it from strict edit wars to all contested page-days. The existing table got the two columns through `ALTER TABLE ... ADD COLUMNS` inside the next MERGE, and a full Gold refresh (`load_type=full`) filled them for every day: `agg_edit_wars` went from 1 row to 18. Power BI only ever sees new columns, never renamed or retyped ones, so the report does not break.

4.3 PII management

The data really does contain personal information. Until 2025, logged-out editors were shown by their IP address, and 874 of the 20,000 rows in our dump sample (4.4 percent) had a raw IP as the editor. Registered usernames are often real names. Newer logged-out edits come from temporary accounts such as `~2026-12345-6`. All rules live in one place, `common/pii_policy.py`, and `pipeline/pii.py` applies the same regexes and the same hash in Spark during the Bronze to Silver step.

Table 6. PII rules applied before data reaches Silver.

Field	Treatment	Result
Registered username	salted SHA-256, first 16 hex characters	user_<hash>
IP address (older logged-out edits)	salted SHA-256	ip_<hash>
Temporary account name	salted SHA-256	tmp_<hash>
Bot and system accounts	kept: public service accounts that must carry "bot" in the name	name unchanged
User and User talk page titles	user part hashed	User:user_<hash>
Edit summaries	IPs, e-mail addresses, User: and Special:Contributions links replaced by regex	[ip], [email], [user]; not copied to Gold
Block and rename targets	hashed like editors	target_key

Hashing, not encryption, is the right tool here because we never need the original value back. The salt matters. There are only about 4.3 billion IPv4 addresses, so an unsalted hash of an IP can be reversed by brute force in minutes. The salt is a long random string in the Databricks secret scope `pkwiki/pii_salt`; it is never in the code or on GitHub. Because the hash is deterministic, the same editor gets the same key in the dump and in the API, so distinct-editor counts, retention and revert-by-a-different-editor all still work. Truncating to 64 bits keeps keys short; with even a million distinct editors the chance of any collision is about 3 in 100 million.

Raw values exist only in Bronze, which only the two of us can read. While building Phase 2 we also found and fixed a leak in our own Phase 1 rules: the API's `bot` flag is set for humans who use the flood flag, so treating it as "bot account" kept a few real usernames unhashed. Now only accounts with "bot" in the name and a short list of system accounts keep their names, and the published sample files were masked again. A check in `99_checks` confirms every editor key is a hash or a bot name and that no IP pattern is left in any masked summary.

4.4 Idempotent MERGE, SCD Type 2 and incremental watermarking

Idempotency. Every table except the audit log is written with `MERGE INTO` [6], and every MERGE has a condition that makes a repeat a no-op. Bronze only inserts lines whose `record_hash` is not yet in the partition. Silver updates a row only when its `row_hash` (a hash of all source attributes) changed, and inserts only new keys. Revert flags only ever move from false to true. Duplicates inside one batch are removed with a window function before the MERGE, preferring the dump over the API and then the newest row. We tested

this on Databricks by running the full load and the daily load twice: the second runs inserted, updated and deleted zero rows in every layer.

Dump and API together. The dump is rebuilt every month and Wikimedia warns against stitching dumps together. So a full load of snapshot `2026-08` takes over everything before 1 September for the wikis in the run. The Silver MERGE adds `WHEN NOT MATCHED BY SOURCE AND t.source = 'api' AND t.wiki IN (...) AND t.rev_ts < cutoff THEN DELETE`. API rows the dump also has are replaced by the dump version, API rows before the cutoff that the dump does not have are removed, and API rows after the cutoff stay. This is the only place the pipeline deletes rows. Deleted pages are never removed; they get `is_deleted = true`.

SCD Type 2. `silver.page` tracks title, namespace, redirect and deleted; `silver.editor` tracks editor type and bot status. For each batch we build a timeline of observations per key, forward-fill attributes the source did not send, hash them into `attr_hash`, and keep only the rows where the hash changes. The write is the usual staged MERGE: a changed key appears twice in the source, once with a merge key that matches and closes the current version (`valid_to`, `is_current = false`) and once with a null merge key that can never match and is inserted as the new current version. A new version is only written when the hash differs and the change is newer than the current version, which keeps reruns and late batches safe. The checks notebook confirms exactly one current row per key and no gaps or overlaps between versions.

Watermarking works at three levels. The extractor keeps a per-wiki watermark in `watermarks.json` and asks the API for `rcstart = watermark` to `rcend = now`. If two windows overlap, the duplicate lines have the same `record_hash` and Bronze ignores them. The standard daily run then processes the two newest `ingest_date` partitions, not one: re-reading yesterday costs almost nothing (the MERGE inserts zero rows) and catches files that landed after yesterday's run. Gold keeps its own watermark per table in `ops.gold_watermark`, set to the highest Silver `load_timestamp` it has consumed. We use the load time and not the event time on purpose. A dump restatement changes rows with old event times, but every changed Silver row gets a new `load_timestamp`, while unchanged rows are not touched at all. So the watermark picks up restatements that an event-time watermark would miss, and nothing else.

4.5 Gold MERGE strategy and table optimisation

Gold runs as the last task of every job run, right after the Silver SCD2 step. The order inside the task matters: the small dimensions first, then `dim_page` and `dim_editor`, then the two event facts (which look up dimension versions), and the aggregates last because they are built from the facts.

Facts and SCD2 dimensions are MERGEd on their surrogate key. The source is only the Silver rows whose `load_timestamp` is newer than the table's watermark, plus any fact rows that still point at an unknown member. A matched row is updated only when its `row_hash` differs (`NOT (t.row_hash <=> s.row_hash)`), so a rerun is a no-op and an unchanged row is never rewritten. The watermark is moved in a separate MERGE only after the data MERGE has succeeded, so a failed run simply reads the same rows again next time. The page and editor version for each edit is found with one equi-join on the natural key plus a window that picks the version whose validity range contains the edit time. We avoided a range join on `valid_from` and `valid_to` because Spark cannot hash-partition on an inequality and would fall back to a much slower nested loop. A full load MERGEs against all of Silver and adds `WHEN NOT MATCHED BY SOURCE THEN DELETE`, so Gold follows the monthly handover.

Aggregates are not MERGEd from a delta. A median or a count of distinct editors cannot be updated by adding new rows to old totals, so we recompute whole groups instead. The run collects the `date_key` values of the fact rows it touched (they carry its `batch_id`), recomputes `agg_wiki_daily` and `agg_edit_wars` for those days and `agg_revert_latency` for those months, and MERGEs the result with a scoped delete such as `WHEN NOT MATCHED BY SOURCE AND t.date_key IN (...) THEN DELETE`. That removes groups that no longer exist without touching any other day. A normal daily run touches only a day or two (the last one on 25 October touched one), so the aggregates cost little. If a backfill touches more than 400 days, we

simply recompute everything, which is cheaper than a huge IN list. This is the same result `replaceWhere` would give, but a MERGE leaves unchanged rows alone and records exact insert, update and delete counts in the audit log.

Optimisation. After the loads, `optimize_gold()` lays out the four tables that grow with the data. On Databricks it uses liquid clustering: `ALTER TABLE ... CLUSTER BY (date_key, wiki_key, page_key)` is set once, and the daily `OPTIMIZE` only clusters files that are not clustered yet, so it stays cheap as the table grows. The open source Delta we use locally has no `CLUSTER BY` for existing tables, so there the same function falls back to `OPTIMIZE ... ZORDER BY`. We chose liquid clustering over partitions plus ZORDER for three reasons: the tables are small enough that partitions would create small files, ZORDER rewrites everything each time while liquid clustering is incremental, and the keys can be changed later without a rewrite. The chosen keys follow the dashboard: almost every visual filters on a date range and a wiki, and the edit-war and page views join on `page_key`. An OPTIMIZE that fails is logged as SKIPPED and never fails the load, because layout only affects speed, not correctness.

What the Databricks runs showed. The current numbers come from one multi-task run on serverless on 25 October, after the key fix in Section 4.6. It first ran a full Gold refresh (`dbx-gold-full-3`, 167 seconds of execution), then a normal daily pass through Bronze, Silver, SCD2 and Gold (`dbx-gold-incr-2`) on a second extractor pull of 168 new changes for the same day. The incremental Gold task took 162 seconds. Table 7 shows what each Gold table looked like before and after that daily pass.

Table 7. Gold row counts on Databricks before and after one daily incremental run.

Gold table	After full refresh	After daily run	Change	How the daily run wrote it
dim_page (SCD2)	13,859	13,886	+27	MERGE of Silver versions newer than the watermark
dim_editor (SCD2)	577	578	+1	MERGE of Silver versions newer than the watermark
fact_revision	32,102	32,144	+42	MERGE of new Silver revisions
fact_page_event	7,786	7,799	+13	MERGE of new Silver page events
agg_wiki_daily	727	727	0	recomputed for the 1 touched day, 3 rows updated
agg_revert_latency	59	59	0	recomputed for the touched month, no change
agg_edit_wars	18	18	0	recomputed for the touched day; 6 rows are strict edit wars
agg_editor_monthly	160	160	0	recomputed, 4 rows updated
dim_date, dim_wiki, dim_change_type	10,957 / 4 / 12	unchanged	0	no-op MERGE

The facts match Silver row for row (Silver `revision` also has 32,144 rows), and each SCD2 dimension is its Silver table plus the four unknown members. The checks found no duplicate surrogate keys, one row per business key in both facts, and zero orphans in all 15 foreign-key checks (a `LEFT ANTI JOIN` of every fact key to its dimension). Active editors in `agg_wiki_daily` match a recount from `fact_revision` on every row we compared. `DESCRIBE DETAIL` lists the clustering columns on all four clustered tables, and OPTIMIZE compacts the files each daily MERGE adds back into one per table.

4.6 A surrogate key bug, found and fixed

The first Gold runs looked fine on row counts, but a key check showed a real bug. `xxhash64` returns a signed 64-bit number, so about half of all hashes are negative. Our code reserves negative keys for the unknown members (`-1` to `-4`), and the point-in-time lookup skips dimension rows with a negative key. As a result 7,060 of 13,859 page versions and 274 of 577 editor versions could never be found, and 62.68 percent of `fact_revision` rows pointed at an unknown page (37.83 percent at an unknown editor). Every incremental run also re-read those rows as "retries", which inflated its input.

The fix is one line in `sk()`: clear the sign bit with a bitwise AND of `0x7FFFFFFFFFFFFFFF`, so every real key is zero or positive. Because about half of the keys changed, we ran a full Gold refresh before any incremental run. The full path re-keyed 7,060 page and 274 editor versions, replaced 16,097 revision facts with re-keyed copies and repointed 9,700 more, and its `WHEN NOT MATCHED BY SOURCE THEN DELETE` removed the rows with old keys. An incremental run first would have matched on the new `revision_key`, found nothing, and inserted duplicates. Afterwards the unknown-page share fell to 23.09 percent, all of it explained (Section 3.1), and the unknown-editor share to zero. The lesson for us was to test key ranges and orphan counts, not only row counts.

5 Spark UI and DAG analysis

This section looks at two Gold queries. The first is a MERGE, captured in the Databricks Query Profile while the Gold task ran (Figure 4). It shows how Delta matches new rows to the rows already in a table. The second is the SELECT that builds `agg_wiki_daily`, the heaviest aggregation, which we read from its `EXPLAIN FORMATTED` plan (Listing 1). Databricks Free Edition is serverless, and serverless compute has no classic Spark UI because there is no cluster behind it. The Query Profile [7] takes its place: it shows the same operator graph, with rows, time and memory for each node.

5.1 Query Profile of a Gold MERGE

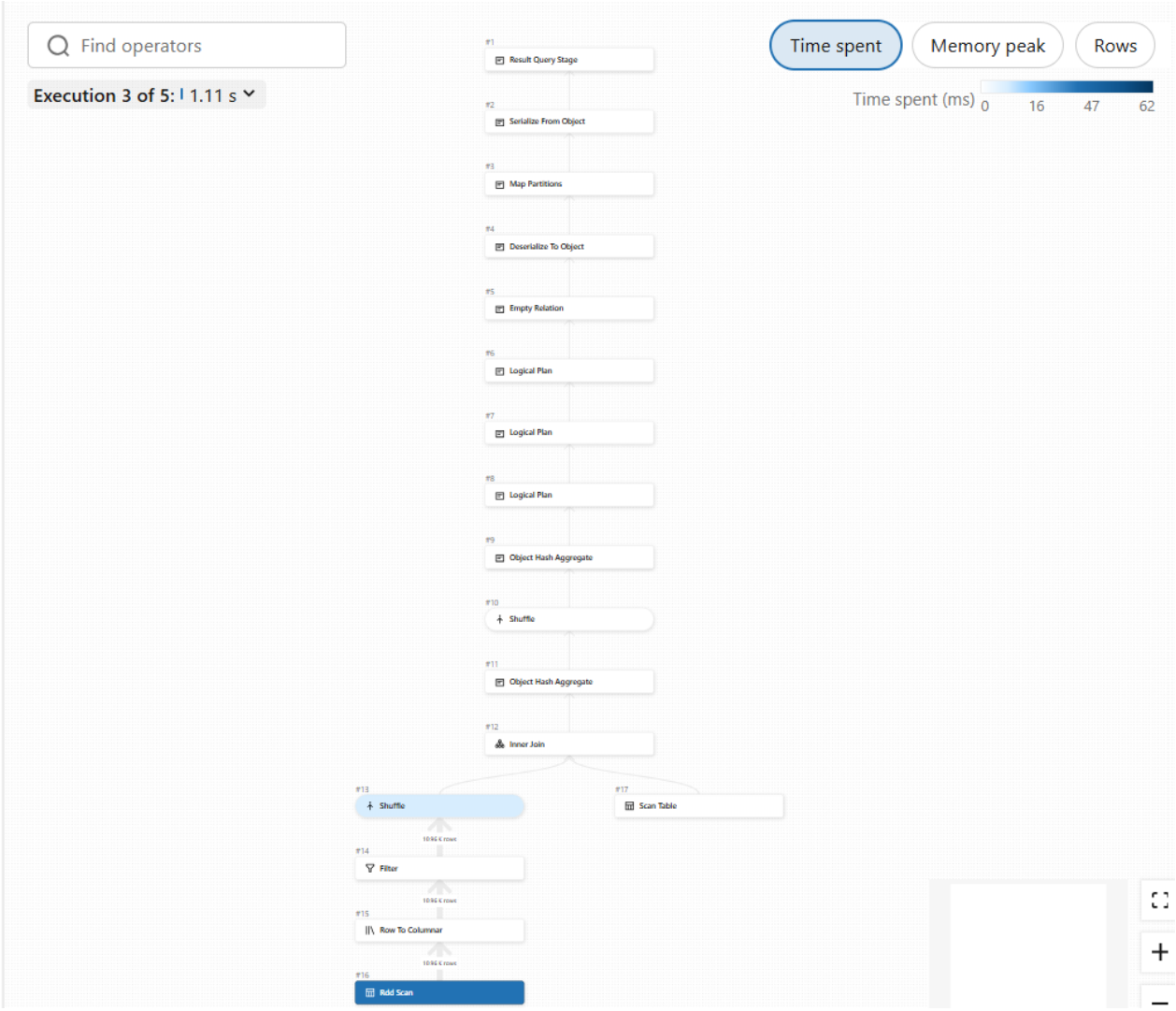


Figure 4. Databricks Query Profile (serverless, graph view) of the heaviest MERGE of a full Gold refresh, the 11.83 s call result = spark.sql(merge_sql), most likely the MERGE into fact_revision. Shown: execution 3 of 5 of that statement (1.11 s). Read from the bottom up.

The profile belongs to the slowest statement of a full Gold refresh, a `spark.sql(merge_sql)` call that took 11.83 seconds. One MERGE statement runs as several Spark executions; this one had five, and the figure shows the third. It is the step where Delta works out which rows of the target table the new data touches. Reading from the bottom up:

- **Source side (left branch).** `Rdd Scan` and `Row To Columnar` read the MERGE source, the batch of changed rows that `merge_gold()` registers as a temporary view. A `Filter` follows. About 11 thousand rows (10.96 K) go in and the same number come out, so nothing is dropped in this run, but the point is where it sits: the predicate is applied before any data moves between tasks.
- **Shuffle into the join.** The `Shuffle` above the filter repartitions the source rows on the MERGE key, so that every source row lands in the same task as the target rows with the same key.
- **Target side (right branch).** `Scan Table` reads the Delta target. Delta does not need every file here. It compares the MERGE condition with the minimum and maximum values it stores for each file and skips the files that cannot match. On another Gold MERGE of the same run the profile reported 27 percent of the files skipped and only 2 percent of the bytes read after the filter.
- **Inner Join.** This join is the heart of the MERGE: it pairs each source row with the target row that has the same key. Rows that find a partner become updates (if their `row_hash` differs). Rows that find none become inserts in a later execution.

- **Two-phase aggregation.** `Object Hash Aggregate`, then a `Shuffle`, then a second `Object Hash Aggregate`. The first one aggregates inside each task (partial), the shuffle brings equal keys together, and the second one finishes the job (final). Delta uses this to count matches per target row, because MERGE must fail if two source rows match the same target row, and to collect the names of the files that contain matches. Only those files get rewritten.
- **Top of the graph.** The three `Logical Plan` nodes and the `Empty Relation` are parts of the plan that the profile keeps in logical form or that were found empty at run time. `Deserialize To Object`, `Map Partitions` and `Serialize From Object` are typed Dataset code inside Delta's MERGE that turns the result into the list of touched files. The `Result Query Stage` returns that small result to the driver.

The whole execution took 1.11 seconds, and the time scale of the profile tops out at 62 milliseconds for a single operator. Most of the 11.83 seconds of the statement is therefore not in this matching step. It goes to the other executions, which rewrite the touched files, and to the Delta commit. That fits what the billing data told us in Section 6: at our volume the cost is mostly fixed overhead per statement, not data processing.

5.2 Physical plan of the agg_wiki_daily build

The second query is the SELECT inside `wiki_daily_frame()` (`pipeline/gold.py`), whose result is then written with `MERGE INTO agg_wiki_daily`. It left-joins `fact_revision` to `dim_editor` on `editor_key`, groups by `(wiki_key, date_key)` and computes about fifteen measures, among them two distinct counts of editors and a median of `seconds_to_revert`. A smaller aggregation over `fact_page_event` counts deletions, restores and moves per wiki and day, and the two results are joined on `(wiki_key, date_key)`. We ran `EXPLAIN FORMATTED` on it on Databricks serverless after the full Gold build.

```
AdaptiveSparkPlan (31)
+- == Initial Plan ==
   PhotonResultStage (30)
   +- PhotonColumnarToRow (29)
      +- PhotonProject (28)
         +- PhotonProject (27)
            +- PhotonShuffledHashJoin FullOuter (26)
               :- PhotonGroupingAgg (19)
               :  +- PhotonShuffleExchangeSource (18)
               :     +- PhotonShuffleMapStage (17)
               :        +- PhotonShuffleExchangeSink (16)
               :           +- PhotonGroupingAgg (15)
               :              +- PhotonGroupingAgg (14)
               :                 +- PhotonShuffleExchangeSource (13)
               :                    +- PhotonShuffleMapStage (12)
               :                       +- PhotonShuffleExchangeSink (11)
               :                          +- PhotonGroupingAgg (10)
               :                             +- PhotonExpand (9)
               :                                +- PhotonProject (8)
               :                                   +- PhotonBroadcastHashJoin LeftOuter (7)
               :                                      :- PhotonScan parquet fact_revision (1)
               :                                      +- PhotonShuffleExchangeSource (6)
               :                                         +- PhotonShuffleMapStage (5)
               :                                            +- PhotonShuffleExchangeSink (4)
               :                                               +- PhotonProject (3)
               :                                                  +- PhotonScan parquet dim_editor (2)
               +- PhotonGroupingAgg (25)
                  +- PhotonShuffleExchangeSource (24)
                     +- PhotonShuffleMapStage (23)
                        +- PhotonShuffleExchangeSink (22)
                           +- PhotonGroupingAgg (21)
                              +- PhotonScan parquet fact_page_event (20)
```

Listing 1. Physical plan of the SELECT that builds agg_wiki_daily, Databricks serverless (EXPLAIN FORMATTED, 25 October 2026, table prefixes shortened). The full output is in docs/databricks_facts.md.

Every operator is a Photon operator, and the plan ends with "The query is fully supported by Photon". Spark splits the query into four map stages and a result stage, with a boundary at every exchange (Table 8).

Table 8. Stages of the agg_wiki_daily SELECT, read from Listing 1.

Stage	Plan nodes	What happens
1	2 to 5	Scan dim_editor (3 columns) and send it to every executor. The exchange is EXECUTOR_BROADCAST, a broadcast rather than a shuffle.
2	1, 6 to 12	Scan fact_revision, only the 13 columns the query needs. Broadcast hash left join with dim_editor (node 7), so the big side is not shuffled for the join. Expand (node 9) emits each row twice for the distinct counts, then a partial aggregation and a shuffle on (wiki_key, date_key, editor_hash, gid).
3	13 to 17	Merge the partial results per editor, then a partial aggregation by (wiki_key, date_key) and a second shuffle on those two columns.
4	20 to 23	Scan fact_page_event (3 columns), partial aggregation, shuffle on (wiki_key, date_key).
Result	18, 19, 24 to 30	Final aggregations of both sides, full outer shuffled hash join on (wiki_key, date_key), then coalesce to zero, bot_edit_share and net_bytes.

Group-by and shuffles. As in the MERGE profile, Spark aggregates in two phases: a partial aggregation in each task, a shuffle of the much smaller partial results, then the final aggregation. The two `countDistinct` measures add a third level. `PhotonExpand` (node 9) emits each row twice, once for the plain sums (`gid = 0`) and once carrying `editor_hash` (`gid = 1`). Grouping by `(wiki_key, date_key, editor_hash, gid)` removes duplicate editors, and grouping again by `(wiki_key, date_key)` counts what is left. That extra shuffle (node 11), with twice the rows, is the most expensive part of the plan.

Joins. `dim_editor` has a few hundred rows, far below the broadcast threshold, so it is broadcast to the executors and joined there (node 7). The final join is a full outer join, because a day can have page events but no edits or the other way round. A full outer join cannot be broadcast, so Spark chose a shuffled hash join (node 26). It needs no new exchange, because both inputs already leave their aggregations hash-partitioned on `(wiki_key, date_key)`.

Pruning and pushdown. Each scan reads only the columns it needs (`ReadSchema`): 13 of `fact_revision` and 3 of `fact_page_event`. This EXPLAIN covers all days, so it has no filter. In the daily run the `date_key IN (...)` predicate is pushed into the scans. Gold has no Hive partitions, so Delta skips files using the per-file minimum and maximum of `date_key`, the same mechanism that skipped 27 percent of the files in the MERGE above, and liquid clustering keeps those ranges narrow. True partition pruning happens earlier in the pipeline: Bronze reads `WHERE ingest_date IN (...) AND wiki IN (...)` and opens only those folders, and Silver prunes on `wiki`.

Adaptive Query Execution. The root node is `AdaptiveSparkPlan` with `isFinalPlan=false`, so this is the plan before it runs. With AQE [8], Spark plans again after each stage using the real sizes of the shuffle outputs. The plan starts with 16 shuffle partitions per exchange, and since the result is only a few hundred rows (727 after the daily run), AQE merges them into a handful of tasks. It could also switch a join to a broadcast or split a skewed partition if the data called for it. The `Empty Relation` in the MERGE profile is the same mechanism at work: a branch that turned out to be empty at run time was dropped.

5.3 How the profile was captured

- Open the run page of the Gold task (Jobs and Pipelines, then the run of `04_gold_star`), or run `notebooks/04_gold_star` again.
- Under the cell `run = runner.run_gold(...)`, open **See performance**. It lists every query the cell ran, with its duration.
- Click the slowest `result = spark.sql(merge_sql)` entry to open its Query Profile, pick the execution in the drop-down at the top, and switch to the graph view.
- Another way in is **Query History** in the sidebar, filtered to serverless compute and the time of the run.
- On a classic cluster the same graph is in the **Spark UI**: open the **SQL / DataFrame** tab, click the query and look at its **DAG**. The **Stages** tab shows one stage per exchange.

6 FinOps

6.1 Pricing basis

Our runs used Databricks Free Edition, which charges nothing but still records usage in the `system.billing.usage` table. So we know how many DBUs (Databricks Units) each run consumed, and we price those DBUs with Microsoft's published pay-as-you-go rates for Azure Databricks Premium in East US, read from the Azure Retail Prices API and the Azure Databricks pricing page on 25 October 2026 [1, 2]. Classic compute is billed twice, once in DBUs and once for the virtual machines [3]. A `Standard_DS3_v2` node (4 vCPU, 14 GiB) is rated at 0.75 DBU per hour. Serverless compute charges one DBU rate that already includes the machines. Azure for Students credit is spent at the same rates.

Table 9. Azure prices used in this section (East US, pay-as-you-go, Premium tier) [1, 2].

Item	Rate (USD)	Unit
Jobs Compute (Premium)	0.30	per DBU
Jobs Compute Photon (Premium)	0.30	per DBU (Photon nodes emit more DBUs per hour)
All-purpose Compute (Premium)	0.55	per DBU
Serverless Jobs (Automated Serverless Compute)	0.45	per DBU, machines included
Serverless SQL warehouse	0.70	per DBU, machines included
Standard_DS3_v2 VM, Linux, on-demand	0.293	per hour
Standard_DS3_v2 VM, Linux, spot	0.0682	per hour (can be evicted)
DBU rating of Standard_DS3_v2	0.75	DBU per node hour

6.2 Typical daily incremental run

Table 10. Daily incremental run dbx-gold-incr-2 on serverless (25 October 2026, current code). Times are measured; DBUs are estimated from the rate measured on 11 October.

Task	Notebook	Setup s	Execution s	Wall s	DBU (estimated)
bronze	01_bronze_ingest	2	56	58	0.183
silver	02_silver_unify_and_mask	1	53	54	0.173
scd2	03_silver_scd2_and_reverts	1	50	51	0.163
gold	04_gold_star	2	162	164	0.529
Total			**321**	**327**	**1.048**

Table 10 is our latest daily run on 25 October (`dbx-gold-incr-2`): Bronze, Silver, SCD2 and Gold as one multi-task run on the current code. Bronze re-read the whole 25 October folder (13,070 change lines) and inserted the 168 that were new. Execution took 321 seconds, about 5.3 minutes, and half of it was the Gold task. The first daily run of that day, before the key fix, inserted 12,902 new changes and took 344 seconds, so the run time barely depends on how much data arrives. The billing table only runs to 15 October, so the DBUs of these runs are not visible yet. We estimate them from the incremental run of 11 October, which used 0.5943 DBU for 182 seconds of execution, or 0.00327 DBU per second. That gives 1.05 DBU for the whole run.

On Azure serverless jobs that is 1.048 x $0.45 = $0.472 per run, or $172.16 for a year of daily runs. At the list price of our own workspace's serverless SKU ($0.35 per DBU) it would be $0.367. On the free tier it cost nothing; all 22 runs of 11 October together used 4.45 DBU.

The same run on a classic single-node job cluster (`Standard_DS3_v2`) is billed for the execution time plus about 5 minutes of cluster start-up. We bill the start-up in full for both the VM and the DBUs, which slightly overstates the cost, and we assume one 4-core node runs the job about as fast as serverless did, since the run is dominated by fixed overhead. Billable time is (5.3 + 5) / 60 = 0.172 h. The DBUs cost 0.75 x 0.172 x $0.30 = $0.039 and the VM $0.293 x 0.172 = $0.051, so one run is $0.089 and a year of daily runs $32.61.

Two things in the measurements surprised us. First, the cost is almost all fixed overhead. The standard daily run that found nothing new (it inserted zero rows) still used 0.50 DBU for Bronze, Silver and SCD2, about 85 percent of what the loaded run of 11 October used for the same three tasks. Delta commits, MERGE planning and the audit writes cost the same whether a day has 17,000 events or none. The Gold task shows the same thing: in the last daily run it took 162 seconds although only 42 new revisions arrived. Second, the SQL warehouse is not a small line. One warehouse session on 11 October used 2.44 DBU, more than half of all job DBUs that day. If Power BI refreshes once a day on a 2X-Small serverless warehouse (about 4 DBU per hour) that auto-stops after 10 idle minutes, each refresh costs about $0.47, or $170 a year, close to what the pipeline itself costs on serverless. That is why the dashboard uses import mode with one scheduled refresh after the job instead of DirectQuery.

6.3 Enterprise extrapolation: hourly runs for a year

Now suppose the pipeline becomes an enterprise service. It covers many more wikis and runs every hour so the dashboard is close to real time, which is 8,760 runs a year. We assume each hourly run handles about as much data as one of our current days. Because most of the measured cost is fixed overhead, the result is not very sensitive to that assumption. Table 11 compares five ways to run it and Figure 5 shows the split between DBU and VM charges.

Table 11. Yearly compute cost of 8,760 hourly runs under different compute options.

Option	Billed h / yr	Basis	DBU cost	VM cost	Total / yr
Always-on all-purpose cluster, 1 node, 24 x 365	8,760	0.75 DBU x 8,760 h x $0.55 + VM 8,760 h x $0.293	$3,614	$2,567	**$6,180**
Job cluster per run, 1 node, on-demand VM	1,511	(5.3 + 5 start-up) min x 8,760 runs, jobs $0.30/DBU	$340	$443	**$783**
Job cluster per run, 1 node, spot VM	1,511	same minutes, VM at spot $0.0682/h; evicted runs retried	$340	$103	**$443**
Serverless jobs, DBUs of the daily run	781	1.048 DBU per run (estimated) x 8,760 x $0.45	$4,132	$0	**$4,132**
Serverless jobs, quiet hours (no new rows)	781	a measured no-op run used 85% of the DBUs of a loaded run	$3,496	$0	**$3,496**

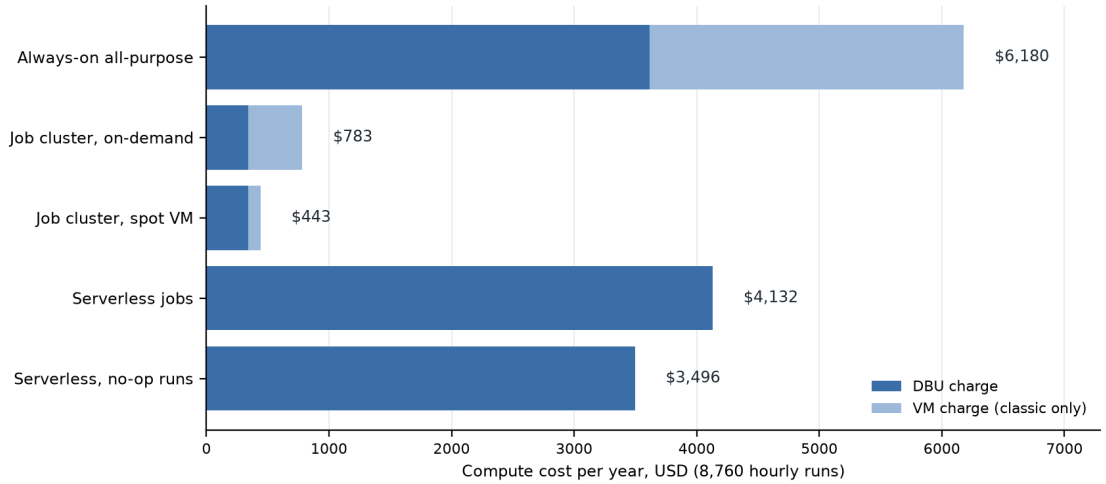


Figure 5. Yearly compute cost of the hourly enterprise scenario, split into DBU and VM charges.

The always-on cluster is the worst choice at $6,180 a year. It pays the all-purpose DBU rate for all 8,760 hours although the work needs about 5 minutes of each hour: 0.75 x 8,760 x $0.55 = $3,614 in DBUs plus 8,760 x $0.293 = $2,567 for the VM. A job cluster that starts for each run and terminates afterwards costs $783, 87 percent less, and a spot VM brings it to $443. Spot capacity can be taken back at short notice, but our MERGEs are idempotent, so an evicted run is simply retried.

Serverless comes out at $4,132, and still $3,496 if most hours bring no new data. We expected the opposite before we had the billing data. For short, overhead-heavy runs, serverless meters many DBUs per run, while a small classic node is cheap per hour and its start-up costs a few cents. Serverless still has real advantages. Our tasks waited only 2 to 6 seconds for their environment instead of minutes, there is nothing to size or patch, and a multi-task job reuses one environment, which cut our wall time by about 20 percent compared with running the notebooks one by one. For an hourly pipeline where a five-minute start-up is acceptable, we would use a job cluster on a spot VM with fallback to on-demand. Where freshness matters more than cost, serverless is the simpler choice.

6.4 Optimisations

Table 12 lists the levers we would use, roughly in the order we would apply them, and what each one does in this pipeline.

Table 12. Cost optimisation levers.

Lever	What it does here	Expected effect
Job clusters with auto-termination	Each run gets its own cluster that stops when the job ends; any interactive cluster auto-terminates after 15 to 30 idle minutes	$6,180 down to $783 a year in the hourly case
Spot or low-priority VMs	Spot for the node, or for the workers with an on-demand driver on bigger clusters; idempotent MERGEs make a retried run safe	VM bill down from $443 to $103 a year
Serverless where latency matters	Seconds to start, nothing idle; one multi-task job reuses one environment	about 20 percent less wall time, but a higher DBU bill for tiny runs
Photon for Gold	Vectorised engine for the joins and group-bys in the Gold build and for Power BI queries (already on for our SQL warehouse)	worth it only when the speed-up beats Photon's higher DBU rate; not for Bronze
Incremental Gold	Facts MERGE only Silver rows newer than the watermark; aggregates recompute only the touched days and months	run time follows the day's changes, not the whole history
File compaction	OPTIMIZE plus optimized writes merge the small files left by daily MERGEs; VACUUM removes old versions weekly	fewer files to list and open on every read
Liquid clustering	Gold facts clustered by date, wiki and page so dashboard filters skip most files; OPTIMIZE only touches new data	less data scanned per query and per MERGE
Right-sizing	Single node for the daily run; more workers only for the monthly full load; smallest SQL warehouse with a short auto-stop	no paying for idle cores
Batched SQL sessions	Power BI import mode with one refresh after the job	about $170 a year instead of a warehouse that rarely stops
Storage lifecycle tiers	Raw landing files to Cool after 30 days, then Archive or delete; old dump snapshots deleted once the next one is in	storage per GB down by up to 95 percent (Table 13)

Table 13. Azure Blob (ADLS Gen2) LRS storage prices, East US [2, 4].

Tier	USD per GB-month	1 TB per month	1 TB per year
Hot	0.0208	$20.80	$249.60
Cool	0.0152	$15.20	$182.40
Cold	0.0036	$3.60	$43.20
Archive	0.00099	$0.99	$11.88

Storage is tiny for us. The Phase 1 estimate was 0.3 to 0.8 GB of Delta tables for all four wikis, which costs cents per month in the Hot tier, and Free Edition recorded only 1.6 storage units over five weeks. Lifecycle rules still matter for the raw files, because they are the part that grows every day and is almost never read again after loading.

7 Conclusion and future work

We set out to build a lakehouse that could tell the Urdu, Punjabi, Sindhi and Pashto Wikipedia communities whether their projects are growing, how quickly vandalism is cleaned up, where edit wars happen and how much is written by bots. The finished pipeline does this from two very different sources. It keeps raw data intact in Bronze, merges and masks it in Silver, and serves a star schema from Gold. Every write is idempotent, every step is audited, and schema changes end up in a log instead of a failed job. The hardest part was the monthly handover between dump and API, and it is handled by one MERGE with a bounded delete.

The cost picture is clear. At our real size the daily run costs about $0.09 on a job cluster or $0.47 on serverless, and nothing on Free Edition. Run every hour for a year, the same pipeline costs $443 to $783 on job clusters and $3,496 to $4,132 on serverless jobs, against $6,180 for an always-on interactive cluster. The billing data also taught us that our runs are dominated by fixed overhead, so the cheapest levers are the ones that cut idle time and per-run overhead, not a faster engine.

The main items left for future work:

- Move the extractor into the job as a first task, with its watermark file in the volume, so nothing runs on a laptop.
- Load the real all-time dumps for all four wikis (about 5 to 10 million events) and repeat the timing and cost measurements at that size.
- Add the DB-IP Lite country lookup for old IP edits from the proposal, then drop the IP hash entirely for those rows.
- Automate deletion of raw landing files 30 days after processing, as the proposal promised.
- Replace the daily API pull with the Wikimedia EventStreams feed and Structured Streaming for near real-time Gold, using the same MERGE logic in `foreachBatch`.
- Alert on FAILED rows in the execution log and on new drift log entries, and track spend with the `system.billing.usage` table once we are on a paid workspace.

8 References

[1] Microsoft. Azure Databricks pricing. https://azure.microsoft.com/en-us/pricing/details/databricks/ (instance DBU ratings and DBU rates; accessed 25 October 2026).

[2] Microsoft. Azure Retail Prices REST API. https://prices.azure.com/api/retail/prices (service 'Azure Databricks' and 'Virtual Machines', region eastus; queried 25 October 2026).

[3] Microsoft. Linux Virtual Machines pricing. https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/.

[4] Microsoft. Azure Blob Storage pricing. https://azure.microsoft.com/en-us/pricing/details/storage/blobs/.

[5] Databricks. Use liquid clustering for tables. https://docs.databricks.com/en/delta/clustering.html.

[6] Databricks. Upsert into a Delta Lake table using merge. https://docs.databricks.com/en/delta/merge.html.

[7] Databricks. Query profile. https://docs.databricks.com/en/sql/user/queries/query-profile.html.

[8] Apache Spark. Performance tuning: Adaptive Query Execution. https://spark.apache.org/docs/latest/sql-performance-tuning.html.

[9] Wikimedia. MediaWiki history dumps. https://wikitech.wikimedia.org/wiki/Data_Platform/Data_Lake/Edits/MediaWiki_history_dumps.

[10] MediaWiki. API:RecentChanges. https://www.mediawiki.org/wiki/API:RecentChanges.